

Azure Observable RAG — Pipeline Demo

Walks one PDF through the four ingestion + retrieval stages and renders the intermediate object at every step:

§	Stage	Module
1	Extract	<code>src.extract</code> (live Document Intelligence call)
2	Chunk	<code>src.chunk</code> (Markdown-header + token-bounded split)
3	Index	<code>src.embed</code> + <code>src.index</code> + <code>src.ingest.chunk_to_search_doc</code> (upsert into <code>kb-chunks</code>)
4	Search	<code>src.search.hybrid_search</code> (BM25 + HNSW + L2 semantic)

Prereqs. `.env` populated with Document Intelligence, Azure OpenAI (embed deployment), and AI Search credentials — typically by running `bash infra/deploy.sh`.

```
In [1]: import json, sys, time
        from pathlib import Path

        REPO_ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
        sys.path.insert(0, str(REPO_ROOT))

        from dotenv import load_dotenv
        load_dotenv(REPO_ROOT / '.env')

        import pandas as pd
        from IPython.display import Markdown

        from src.extract import extract
        from src.chunk import chunk_document
        from src.embed import embed_batch
        from src.index import create_or_update_index, get_search_client
        from src.ingest import chunk_to_search_doc
        from src.search import hybrid_search
```

0 · Sample input

We use the Meraki MX67 installation guide — small, well-structured, exercises both DI markdown and figures.

```
In [2]: LOCAL_PDF = REPO_ROOT / 'data/devices/network_access/meraki_mx67/manuals/MX67_MX68_Installat
        BLOB_PATH = 'devices/network_access/meraki_mx67/manuals/MX67_MX68_Installat

        print(f'file: {LOCAL_PDF.name}')
```

```
print(f'size: {LOCAL_PDF.stat().st_size / 1024:.1f} KB')
print(f'blob: {BLOB_PATH}')
```

```
file: MX67_MX68 Installation Guide.pdf
size: 299.0 KB
blob: devices/network_access/meraki_mx67/manuals/MX67_MX68 Installation Guide.pdf
```

1 · Extract — `src.extract.extract`

Calls Document Intelligence (`prebuilt-layout`) on the PDF. Returns

`ExtractedDocument` with markdown, structured tables, figure bounding boxes, and metadata derived from the blob path.

```
In [3]: doc = extract(LOCAL_PDF, BLOB_PATH)

print(f'page_count : {doc.page_count}')
print(f'tables      : {len(doc.tables)}')
print(f'figures     : {len(doc.figures)}')
print(f'markdown    : {len(doc.markdown):,} chars')
```

```
page_count : 13
tables      : 4
figures     : 8
markdown    : 15,005 chars
```

Path-derived metadata — parsed deterministically from the blob path by

`extract._parse_path_metadata` . These fields end up on every chunk and become filterable columns in the AI Search index.

```
In [4]: pd.DataFrame([
    {'scope': doc.scope,
     'device_family': doc.device_family,
     'device': doc.device,
     'doc_type': doc.doc_type,
     'topic': doc.topic,
     'version': doc.version,
     'is_shared': doc.is_shared,
     'file_type': doc.file_type,
    }])
```

```
Out [4]:
```

	scope	device_family	device	doc_type	topic	version	is_shared	file
0	device	network_access	meraki_mx67	manual	mx67- mx68- installation- guide	None	False	

Markdown preview — first ~1500 chars of `doc.markdown` . Note the `<!--`

`PageNumber=N -->` page markers that `chunk.py` will use to attribute each chunk to a source page.

```
In [5]: Markdown(doc.markdown[:1500] + ('\n\n...truncated_' if len(doc.markdown) > 1500))
```

Out [5]: CISCO

Meraki

MX67/MX68 Installation Guide

This document describes how to install and set up the MX67 and MX68 security appliance. Additional reference documents are available online at: www.meraki.com/library/products.

MX67/MX68 Overview

The Meraki MX67 and MX68 are enterprise security appliances designed for distributed deployments that require remote administration. It is ideal for network administrators who demand both ease of deployment and a state-of-the-art feature set. A full overview of the appliances' features can be found in the MX67 and MX68 Overview and Specifications.

Package Contents

In addition to the MX device, the following are provided:

MX67/MX68

MX67W/MX68W

Power Adapter (No Power Cable)

Power Adapter (No Power Cable)

2x CAT5e Ethernet Cables

2x CAT5e Ethernet Cables

2x WiFi Antennae

MX67C MX68CW

Power Adapter (No Power Cable)

Power Adapter (No Power Cable)

2x CAT5e Ethernet Cables

2x CAT5e Ethernet Cables

2x LTE Antennae

2x Attached (Non-Removeable) Hybrid WiFi+LTE Antennae

Front Panels

MX67/67C/67W

MX68/68W/68CW

-

Status Indicator

The MX67/MX68 series devices uses an LED to inform the user of the device's status. LED patterns and their meanings are described below.

LED Status	Meaning
Solid orange	Power is applied but the appliance i ...truncated

```
In [6]: if doc.tables:
        tables_df = pd.DataFrame([
            {'page': t.page, 'columns': len(t.headers), 'rows': len(t.rows), 'headers': t.headers}
            for t in doc.tables[:5]
        ])
        display(tables_df)
    else:
        print('no tables extracted')
```

	page	columns	rows	headers
0	1	2	1	[,]
1	2	2	6	[Field, Value]
2	12	3	1	[Product, Warranty Period, Warranty Information]
3	13	3	2	[Product, Warranty Period, Warranty Information]

```
In [7]: if doc.figures:
        figures_df = pd.DataFrame([
            {'figure_id': f.figure_id, 'page': f.page, 'polygon': [round(x, 2) for x in f.polygon]}
            for f in doc.figures[:5]
        ])
```

```

    ])
    display(figures_df)
else:
    print('no figures extracted')

```

	figure_id	page	polygon
0	p2_f1	2	[0.55, 8.45, 7.9, 8.45, 7.91, 9.36, 0.55, 9.36]
1	p3_f1	3	[0.56, 1.36, 7.92, 1.36, 7.92, 5.85, 0.56, 5.84]
2	p4_f1	4	[0.57, 0.71, 7.9, 0.71, 7.9, 4.24, 0.57, 4.23]
3	p5_f1	5	[0.49, 3.09, 7.93, 3.09, 7.92, 6.14, 0.49, 6.14]
4	p6_f1	6	[0.56, 0.7, 7.92, 0.7, 7.92, 4.68, 0.56, 4.67]

2 · Chunk — src.chunk.chunk_document

Two-stage split: by Markdown headers first (so each chunk carries its `heading_path`), then token-bounded via `tiktoken` (default `CHUNK_MAX_TOKENS=512`). Each chunk gets a deterministic content-hashed `chunk_id` so re-ingestion is idempotent.

```

In [8]: chunks = chunk_document(doc)

lengths = [len(c.content) for c in chunks]
print(f'chunks          : {len(chunks)}')
print(f'content (chars): mean {sum(lengths)//len(lengths)} min {min(lengths)}')
print(f'distinct heading paths: {len({c.heading_path for c in chunks})}')

```

```

chunks          : 31
content (chars): mean 466 min 13 max 2145
distinct heading paths: 30

```

```

In [9]: chunks_df = pd.DataFrame([
    'idx':          c.chunk_index,
    'chunk_id':     c.chunk_id[:12] + '...',
    'page':         c.page_number,
    'heading_path': c.heading_path,
    'preview':      c.content[:120].replace('\n', ' ') + ('...' if len(c.content) > 120 else '')
] for c in chunks])
chunks_df

```

Out [9]:

	idx	chunk_id	page	heading_path	preview
0	0	32e74e4eba12...	1	NaN	CISCO Meraki
1	1	4643575b6784...	1	MX67/MX68 Installation Guide	# MX67/MX68 Installation Guide This document ...
2	2	e6867c82cc43...	1	MX67/MX68 Overview	# MX67/MX68 Overview The Meraki MX67 and MX68...
3	3	2199210300e2...	1	Package Contents	# Package Contents In addition to the MX devi...
4	4	7e7ff08710ab...	1	Front Panels	# Front Panels MX67/67C/67W
5	5	9f040c7a04c9...	2	MX68/68W/68CW	# MX68/68W/68CW \-
6	6	82824a8c11ca...	2	MX68/68W/68CW > Status Indicator	## Status Indicator The MX67/MX68 series devi...
7	7	2d3426c9803f...	2	Back Panels	# Back Panels MX67 <figure> </figure> ...
8	8	1cafdbdfe719...	4	Back Panels > MX67/MX67W/MX67C Back Panel Func...	## MX67/MX67W/MX67C Back Panel Functions Addi...
9	9	fe6ca230a7e2...	6	Back Panels > MX68/MX68W/MX68CW Back Panel Fun...	## MX68/MX68W/MX68CW Back Panel Functions Add...
10	10	62d5bcc8a1f7...	7	Side Panels	# Side Panels MX68 <figure> □ ---- W </f...
11	11	18cb0e7e8424...	8	Side Panels > MX68/MX68W/MX68CW Side Panel Fun...	## MX68/MX68W/MX68CW Side Panel Functions Add...
12	12	6ea3404f0a3c...	8	Side Panels > Bottom Panel	## Bottom Panel <figure> □ □ □ □ </fi...
13	13	9819c276f001...	9	Pre-install Preparation	# Pre-install Preparation You should complete...
14	14	76aab519abee...	9	Configure your Dashboard Network	# Configure your Dashboard Network The follow...
15	15	3bd016884149...	9	Check and Set Firmware	# Check and Set Firmware To ensure your MX pe...
16	16	864e7cbb046d...	10	Check and Configure Upstream Firewall Settings	# Check and Configure Upstream Firewall Settin...

	idx	chunk_id	page	heading_path	preview
17	17	5f5d650c2b60...	10	Check and Configure Upstream Firewall Settings...	## Mounting Hardware The supplied wall screws...
18	18	84a5bafb8a25...	10	Connecting to WAN	# Connecting to WAN All Meraki MX devices mus...
19	19	12246e609ea1...	10	Setting up a Static IP Address	# Setting up a Static IP Address 0 To ensure ...
20	20	398f83119b4e...	10	Setting up a DHCP IP Address	# Setting up a DHCP IP Address By default all...
21	21	072992a098c3...	11	Setting up Cellular Failover	# Setting up Cellular Failover The MX67C and ...
22	22	7d999a47b85d...	11	Setting up Cellular Failover	9\ . Test the cellular failover connection by u...
23	23	4b86dad1a753...	11	Additional Settings	# Additional Settings
24	24	a260f09f3b58...	11	Setting VLANs	# Setting VLANs If your WAN uplink is on a tr...
25	25	d83863e46878...	12	Setting VLANs > Setting up a Secondary WAN int...	## Setting up a Secondary WAN interface on the...
26	26	2c0a88ae0aa6...	12	Setting VLANs > Setting up a Secondary WAN Int...	## Setting up a Secondary WAN Interface on the...
27	27	ce723943b6c9...	12	Setting VLANs > Setting PPPoE	## Setting PPPoE PPPoE authentication may be ...
28	28	f06cddbdb767...	12	Setting VLANs > Setting PPPoE > Web Proxy Sett...	### Web Proxy Settings These settings take ef...
29	29	d3c5cff2a8de...	12	Setting VLANs > Setting PPPoE > Web Proxy Sett...	#### Configuring Physical Link Settings To co...
30	30	39f5a7c29015...	12	Warranty	# Warranty MX Warranty coverage periods are a...

```
In [10]: per_page = pd.Series([c.page_number for c in chunks]).value_counts().sort_ir
per_page.rename_axis('page').to_frame('chunks')
```


Out[10]:

chunks	
page	
1	5
2	3
4	1
6	1
7	1
8	2
9	3
10	5
11	4
12	6

3 · Index — embed + upsert into kb-chunks

Three steps:

1. **Ensure the index exists** (`create_or_update_index` is idempotent).
2. **Embed each chunk's content** via `text-embedding-3-small` (1536-dim).
3. **Shape into search docs** (using `chunk_to_search_doc` — the same helper the Function App's blob trigger uses) and call `upload_documents` .

Idempotent: `chunk_id` is content-hashed, so re-running this cell is a semantic no-op.

```
In [11]: index_name = create_or_update_index()
search_client = get_search_client()
doc_count_before = search_client.get_document_count()
print(f'index          : {index_name}')
print(f'docs before this : {doc_count_before:,}')
```

```
index          : kb-chunks
docs before this : 852
```

```
In [12]: vectors = embed_batch([c.content for c in chunks])
print(f'vectors          : {len(vectors)}')
print(f'embedding dim    : {len(vectors[0])}')
print(f'preview [0][:5]: {[round(x, 4) for x in vectors[0][:5]]}')
```

```
vectors          : 31
embedding dim    : 1536
preview [0][:5]: [0.0303, -0.0351, -0.0057, 0.011, 0.0071]
```

First search-doc as JSON — this is the exact shape that goes into AI Search.

`tables_json` / `figures_json` are stringified because the index schema flattens

them; `content_vector` carries the embedding.

```
In [13]: search_docs = [chunk_to_search_doc(c, v) for c, v in zip(chunks, vectors)]
preview = dict(search_docs[0])
preview['content_vector'] = f'<{len(preview["content_vector"])}-d float[]>'
preview['content'] = preview['content'][:200] + ('...' if len(preview['
print(json.dumps(preview, indent=2, ensure_ascii=False))

{
  "chunk_id": "32e74e4eba121c1ccab7a601633dc5222f5f322e9f212db70aee52d1c4688
dfd",
  "chunk_index": 0,
  "content": "CISCO\n\nMeraki",
  "doc_id": "b97e95bd5b073463",
  "source_path": "devices/network_access/meraki_mx67/manuals/MX67_MX68 Insta
llation Guide.pdf",
  "file_name": "MX67_MX68 Installation Guide.pdf",
  "file_type": "pdf",
  "category": "manual",
  "page_number": 1,
  "page_end": 1,
  "heading_path": null,
  "scope": "device",
  "device_family": "network_access",
  "device": "meraki_mx67",
  "doc_type": "manual",
  "topic": "mx67-mx68-installation-guide",
  "version": null,
  "is_shared": false,
  "tables_json": "",
  "figures_json": "",
  "content_vector": "<1536-d float[]>"
}
```

```
In [14]: result = search_client.upload_documents(search_docs)
succeeded = sum(1 for r in result if r.succeeded)
print(f'upserted: {succeeded}/{len(search_docs)}')

time.sleep(2) # AI Search commit lag
doc_count_after = search_client.get_document_count()
print(f'docs after : {doc_count_after:,} (delta: {doc_count_after - doc_cou

upserted: 31/31
docs after : 852 (delta: +0)
```

4 · Search — `src.search.hybrid_search`

Pure retrieval — no LLM. `hybrid_semantic` mode runs **two stages** inside Azure AI Search; the LangGraph DAG calls this exact function under the hood.

BM25 → RRF fusion → top ~50 → cross-encoder reranker →

top_k
HNSW —

Stage 1 — BM25 + HNSW, fused by RRF

Component	Where it's defined	Notes
BM25 lexical	Azure built-in on any <code>searchable=True</code> field; no code config.	Triggered by passing the query string as <code>search_text</code> .
HNSW vector ANN	<code>src/index.py</code> — <code>HnswAlgorithmConfiguration(m=4, ef_construction=400, ef_search=500, metric=COSINE)</code> , bound to <code>content_vector</code> .	Triggered by passing <code>vector_queries=[VectorizedQuery(...)]</code> in <code>src/search.py</code> .
RRF fusion	Azure built-in default; formula <code>score = $\sum 1/(60 + \text{rank}_i)$</code> with fixed <code>k=60</code> .	Implicit — Azure auto-fuses whenever both <code>search_text</code> and <code>vector_queries</code> are in the same call. We don't write or configure it.

Stage 2 — cross-encoder reranker

Component	Where it's defined	Notes
L2 semantic reranker	<code>src/index.py</code> — <code>SemanticConfiguration(title_field=heading_path, content_fields=[content], keywords_fields=[category])</code> .	Activated per-query with <code>query_type=QueryType.SEMANTIC</code> in <code>src/search.py</code> . Returns <code>reranker_score</code> in the <code>semantic_captions</code> field.

The reranker is a **pretrained transformer cross-encoder** (Microsoft Bing-derived, multilingual — exact architecture not published). Unlike RRF (which only sees rank positions) or cosine (which only sees embedding distance), the cross-encoder reads query + document **together** through attention — scoring actual semantic relevance, not just lexical or numeric proximity.

Other modes — `bm25`, `vector`, `hybrid` — drop one or more stages; see `SUPPORTED_MODES` in `src/search.py`.

```
In [15]: QUERY = 'How do I mount the Meraki MX67 to a wall?'
results = hybrid_search(QUERY, top_k=5, search_mode='hybrid_semantic')

indexed_ids = {c.chunk_id for c in chunks}
results_df = pd.DataFrame([
    {'rank': r.rank,
     'score': round(r.score, 4),
     'reranker_score': round(r.reranker_score, 4) if r.reranker_score is not None else None,
     'page': r.page_number,
     'heading_path': r.heading_path,
     'caption': (r.semantic_captions or '')[:90],
```

```

    'chunk_id':      r.chunk_id[:12] + '...',
    'from_this_run': r.chunk_id in indexed_ids,
} for r in results])
results_df

```

Out[15]:

	rank	score	reranker_score	page	heading_path	caption	chunk_id
0	1	0.0182	2.7430	11	Product Overview > Others	## Others . Paper eject position are selectab...	ccd34f373dc3...
1	2	0.0313	2.6593	1	MX67/MX68 Installation Guide	# MX67/MX68 Installation Guide This document ...	4643575b6784...
2	3	0.0242	2.4558	9	Configure your Dashboard Network	# Configure your Dashboard Network The follow...	76aab519abee...
3	4	0.0238	2.3265	10	Check and Configure Upstream Firewall Settings...	## Mounting Hardware The supplied wall screws...	5f5d650c2b60...
4	5	0.0137	2.2215	13	Accessories > Options	## Options . Wireless LAN cable set (Model: O...	e79f7b116bba...

Off-topic contrast. Same `top_k`, same mode — but a query the document doesn't answer. Expect noticeably lower `reranker_score` values.

```

In [16]: OFF_TOPIC = 'How do I configure a payment terminal pin pad?'
off = hybrid_search(OFF_TOPIC, top_k=5, search_mode='hybrid_semantic')
off_df = pd.DataFrame([
    'rank':      r.rank,
    'reranker_score': round(r.reranker_score, 4) if r.reranker_score is not
    'file_name':    r.file_name,
    'page':         r.page_number,
    'heading_path': r.heading_path,
} for r in off])
off_df

```

Out[16]:

	rank	reranker_score	file_name	page	heading_path
0	1	2.2438	Desk5000 and 3000 series - User guide.pdf	8	Desk Series > CAUTION
1	2	2.2024	Desk5000 and 3000 series - User guide.pdf	8	Desk Series > CAUTION > 3.4 Desk series : Fixe...
2	3	2.0300	tm-m30ii_trg_en_reva.pdf	72	Controlling the Cash Drawer
3	4	2.0157	tm-m30ii_trg_en_reva.pdf	65	Restore Default Values Mode > 1 After running ...
4	5	1.9817	Desk5000 and 3000 series - User guide.pdf	11	Desk Series > 4.Installation > 4.1 Positioning...

5 · Recap

- **Extract** — [src/extract.py](#) → `ExtractedDocument(markdown, tables, figures, +path metadata)`
- **Chunk** — [src/chunk.py](#) → `list[Chunk]` (header + token-bounded split, content-hashed IDs)
- **Index** — [src/embed.py](#) + [src/index.py](#) + [src/ingest.py](#) `.chunk_to_search_doc` → upsert into `kb-chunks`
- **Search** — [src/search.py](#) `.hybrid_search` → `list[RetrievalResult]`

Run the same pipeline over the whole `data/` corpus: `python -m src.ingest`.

See it end-to-end (with the LangGraph DAG + LLM generation): `bash infra/start_chainlit.sh`.